VisionCoder: Reinforcement Learning with Visual Rewards for Screenshot-to-HTML Generation

Amal Joe Job J

{amaljoe88}@gmail.com

Abstract

We present VisionCoder, a system for generating HTML/CSS code from webpage screenshots by fine-tuning a 2-billion-parameter vision-language model with reinforcement learning from verifiable visual rewards. The core technical challenge is that standard language modelling objectives do not optimise for *rendered visual appearance* — a model can produce syntactically valid but visually empty HTML and incur no training loss. We address this by rendering each generated HTML page in a headless browser and comparing the screenshot to the reference image via CLIP visual similarity, using this as the primary reward signal (weighted 3×) alongside three auxiliary rewards for code format, HTML validity, and structural DOM similarity. All rewards are computed automatically without human annotation. We further adopt a resolution curriculum during supervised fine-tuning (SFT): training 60% of steps at quarter resolution (≈12 image tokens), 20% at half resolution (≈48 tokens), and 20% at full resolution (≈192 tokens), before transitioning to Group Relative Policy Optimisation (GRPO) fine-tuning at full resolution. On the Design2Code benchmark (484 real-world webpages, 16), VisionCoder-SFT+RL achieves an overall score of **0.802**, matching Gemini Pro Vision (0.801) and surpassing GPT-4V (0.796) while using > **99**% fewer parameters. Critically, we show that SFT warm-starting is essential: RL trained directly from the base model reaches only 0.354 because the policy collapses into degenerate attractors. Detailed analysis of training dynamics reveals a characteristic generation-length lifecycle, a visual awakening phase transition at step ≈100, and differential entropy responses to visual versus structural reward signals.

1 Introduction

Modern software development is increasingly visual: designers produce high-fidelity mockups and screenshots, and front-end engineers must translate these into HTML, CSS, and JavaScript. This translation is time-consuming, error-prone, and requires detailed knowledge of both visual design and web standards. Automating even part of this workflow — producing a first-pass HTML implementation from a screenshot — would substantially reduce the cost of front-end development.

Vision-language models (VLMs) have demonstrated impressive capabilities on many visual understanding tasks, including image captioning, visual question answering, and document understanding [1, 4, 8]. Screenshot-to-code generation has emerged as a natural test of these capabilities: it requires the model to simultaneously parse the visual layout, identify UI elements and their spatial relationships, and generate syntactically correct, semantically meaningful markup [7, 16].

However, standard fine-tuning approaches face a fundamental limitation: the cross-entropy language modelling objective optimises token-level prediction accuracy, not the visual fidelity of the rendered

output. A model can learn to generate plausible-looking HTML — correctly formatted, with proper tags — yet produce a page that renders as an empty white rectangle. There is no gradient signal pushing the policy toward visual correctness unless the model is explicitly rewarded for what the generated code *looks like* when executed.

We address this gap by casting screenshot-to-HTML generation as a reinforcement learning problem with verifiable visual rewards. Rather than comparing predicted and reference tokens, we *render* the predicted HTML using a headless Playwright browser [9] and compare the resulting screenshot to the target using CLIP image embeddings [12]. This closing of the render loop provides a direct, differentiable signal toward visual fidelity. We call our system VisionCoder.

The core training pipeline follows the pattern established by DeepSeek-R1 [5] for reasoning tasks: begin with supervised fine-tuning (SFT) on a curated dataset to establish a strong initial policy, then apply GRPO [15] to refine the policy toward a multi-component verifiable reward. Critically, we show that this two-stage approach is essential for visual generation: RL from the raw base model reaches only 0.354 on Design2Code, while SFT+RL achieves 0.802 — a 2.3× improvement attributable entirely to the SFT warm-start.

Contributions. We make the following contributions:

- **A multi-component rendering-based reward** combining code format, HTML validity, DOM structural similarity, and CLIP visual similarity (3× weight), all computed automatically without human labels. The theoretical reward ceiling is 6.0; trained models reach ≈ 5.1 in-distribution.
- **A resolution curriculum for SFT** that trains the model progressively from coarse (128 px, ≈12 image tokens) to full resolution (512 px, ≈192 image tokens), reducing early-stage compute by up to 16× while letting the model first learn HTML structure before visual fidelity.
- **Competitive results at 2B scale:** VisionCoder-SFT+RL achieves an overall score of 0.802 on Design2Code, matching Gemini Pro Vision and surpassing GPT-4V, and outperforming GPT-4V by 41% on block-level layout matching.
- **Detailed training dynamics analysis** identifying a four-phase generation-length lifecycle, a visual awakening phase transition, and the mechanism by which SFT warm-starting prevents policy collapse.

2 Related Work

2.1 Screenshot-to-Code Generation

The task of generating UI code from visual inputs has a long history. Beltramelli [3] introduced the problem with a CNN–RNN model trained on synthetic GUI screenshots, achieving 77% accuracy on simple iOS, Android, and web layouts. While pioneering, this approach was limited to constrained domain-specific languages and toy layouts.

HuggingFace M4 Team [7] scaled the problem to real HTML/CSS by releasing WebSight, a dataset of 2 million rendered screenshots paired with their generating HTML. They trained Sightseer, a VLM fine-tuned on this data, demonstrating that large-scale synthetic data can teach models to generate real web markup. Our work builds directly on this dataset.

Figure 1: Overview of the VisionCoder training pipeline. Starting from Qwen3-VL-2B-Instruct, we first apply SFT with a progressive resolution curriculum on 20,000 WebSight samples. The resulting checkpoint is then fine-tuned with GRPO using four verifiable reward components. During GRPO training, each generated HTML is rendered in a headless browser and compared to the reference screenshot via CLIP, providing a direct visual fidelity gradient.

The Design2Code benchmark [16] provides 484 real-world webpages from Common Crawl and establishes a rigorous, rendering-based evaluation protocol that compares predicted and reference pages at the level of detected text blocks, their positions, colours, and overall CLIP similarity. This benchmark, and its OCR-free evaluation pipeline, forms the primary evaluation in our work. The original paper found that GPT-4V was the top-performing model at the time of publication, with an overall score of 0.796, and that a fine-tuned 18B model (Design2Code-18B) based on CogAgent could match Gemini Pro Vision in overall performance.

2.2 Reinforcement Learning from Verifiable Rewards

The use of reinforcement learning to fine-tune language models has a rich history beginning with RLHF [10], which used human preference annotations and PPO [14] to align GPT-3 with human intent. The InstructGPT work demonstrated that even a 1.3B model aligned with RLHF could outperform an unaligned 175B model on human preference tasks.

Subsequent work has moved toward *verifiable* rewards — signals that can be computed automatically

from ground-truth answers without human annotation. DeepSeekMath [15] introduced Group Relative Policy Optimisation (GRPO), an efficient alternative to PPO that eliminates the critic network by using within-group reward statistics to estimate advantages. GRPO has since become widely adopted for its simplicity and efficiency.

DeepSeek-R1 [5] showed that GRPO with verifiable rewards — using rule-based correctness checks for mathematics — can dramatically improve reasoning capabilities, achieving state-of-the-art results on AIME with a 71% solve rate. Crucially, the DeepSeek-R1 work demonstrated both a pure-RL approach (R1-Zero, trained from scratch with RL) and a warm-start approach (R1, with SFT cold-start before RL). The warm-start approach consistently outperformed pure RL, a finding that our work corroborates in the visual generation domain.

DPO [13] offers an alternative to RL-based alignment by reframing preference learning as a classification objective, avoiding explicit reward models. While simpler to train, DPO requires paired preference data and does not naturally accommodate the rendering-based rewards central to our work.

2.3 Vision-Language Models

Large-scale VLMs have been the primary tool for multimodal understanding tasks. LLaVA [8] demonstrated that connecting a pre-trained visual encoder to a large language model and fine-tuning on instruction-following data yields strong multimodal performance. InstructBLIP [4] introduced query-adaptive visual feature extraction, enabling more flexible visual reasoning. Flamingo [1] pioneered in-context few-shot learning for VLMs with interleaved visual-text sequences.

Qwen-VL [2] introduced a competent open-weight VLM with strong spatial understanding and text-reading capabilities, particularly relevant for web screenshots that contain structured text layouts. Qwen3-VL [11] — the base model in our work — extends this with a DeepStack visual architecture and interleaved Multi-scale RoPE positional encodings, supporting dynamic resolution processing and enabling efficient tokenisation at multiple image resolutions.

2.4 Visual Similarity as a Reward Signal

CLIP [12] demonstrated that contrastive pre-training on 400 million image-text pairs yields visual representations that capture semantic and perceptual similarity. While originally designed for image-text alignment, CLIP embeddings have proven effective for image-image similarity, forming the basis of the visual reward in our system.

Zhang et al. [19] showed that deep neural network features outperform pixel-level metrics such as SSIM for perceptual similarity evaluation. This insight motivates our use of high-level CLIP features rather than pixel comparison for reward computation. Using semantically meaningful representations ensures that the reward signal captures visual fidelity at the level of layout and content, not pixel-perfect reproduction.

3 Methodology

3.1 Task Formulation

Given a screenshot $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$ of a webpage, the task is to generate an HTML/CSS string h such that rendering h in a browser produces a page that visually matches $\mathbf{I}$. Formally, we model a policy π_θ that maps a prompt $x = [\text{sys}, \mathbf{I}, \text{query}]$ to a completion $h \sim \pi_\theta(\cdot \mid x)$.

The system prompt instructs the model to act as an expert web developer and to output the complete HTML with inline CSS, using `rick.jpg` as a placeholder for any images. The user query is: *"Generate the HTML code that reproduces this website screenshot."*

Following Qwen3-VL's instruction format, the model is prompted with thinking mode disabled, as chain-of-thought tokens would consume generation budget without contributing to the HTML output.

3.2 Base Model

We train two model scales. Our primary model builds on Qwen3-VL-2B-Instruct [11], a 2-billion-parameter vision-language model with a DeepStack visual architecture. We additionally train a 4-billion-parameter variant using Qwen3-VL-4B-Instruct as the base, following the identical SFT+RL pipeline, to investigate how the approach scales with model capacity. Images are tokenised with patch size 16 and spatial merge factor 2 (effective patch resolution: 32×32 pixels per token). At our training resolution of 512 px maximum width, a typical webpage screenshot occupying roughly 512×384 pixels requires approximately 192 image tokens — a compact but semantically sufficient representation for webpage layout understanding.

The model uses interleaved Multi-scale Rotary Position Embeddings (MRoPE) to encode the two-dimensional spatial structure of image patches alongside one-dimensional text positions, enabling the decoder to reason about the spatial arrangement of visual elements.

3.3 Multi-Component Reward Design

The total reward at each GRPO step is a weighted sum of four components:

$$R = R_{\text{fmt}} + R_{\text{val}} + R_{\text{struct}} + 3 \cdot R_{\text{clip}}, \tag{1}$$

giving a theoretical maximum of $1 + 1 + 1 + 3 = 6.0$. The $3\times$ weighting on R_{clip} makes visual fidelity the dominant signal (50% of maximum reward weight), with the remaining components serving as structural floor constraints that encourage syntactic validity before semantic quality.

Format Reward (R_{fmt}, max 1.0). This binary-style reward checks whether the output conforms to the expected response structure:

$$\begin{aligned} R_{\text{fmt}} = {} & 0.5 \cdot \mathbf{1}[\text{```}\texttt{html} \text{ fence present}] \\ & + 0.25 \cdot \mathbf{1}[\texttt{<html}/\texttt{<!DOCTYPE} \text{ tag present}] \\ & + 0.25 \cdot \mathbf{1}[\texttt{</html>} \text{ closing tag present}]. \end{aligned}$$

This reward converges to near-perfect (≥ 0.99) within the first 50 GRPO steps, indicating that the model quickly learns the structural contract.

Validity Reward (R_{val}, max 1.0). HTML is parsed with the `html5lib` tolerant parser. Three sub-scores assess:

$$\begin{aligned} R_{\text{val}} = {} & 0.4 \cdot \mathbf{1}[\text{parse succeeds}] \\ & + 0.3 \cdot \tfrac{1}{3}\big(\mathbf{1}[\texttt{<html>}] + \mathbf{1}[\texttt{<head>}] + \mathbf{1}[\texttt{<body>}]\big) \\ & + 0.3 \cdot \min\big(\tfrac{|\mathcal{T} \cap \mathcal{K}|}{6}, 1\big), \end{aligned}$$

where $\mathcal{T}$ is the set of distinct tags in the output and $\mathcal{K}$ is a set of 16 key structural tags (`div`, `p`, `span`, `a`, `table`, etc.). The diversity component prevents degenerate outputs that technically parse but contain only a single tag type.

Structural Reward (R_{struct}, max 1.0). DOM structural similarity is measured by comparing the tag-level sequence of the prediction against the reference HTML:

$$R_{\text{struct}} = \frac{2 \cdot \text{LCS}(S_{\text{pred}}, S_{\text{ref}})}{|S_{\text{pred}}| + |S_{\text{ref}}|}, \tag{2}$$

where S denotes the BFS-ordered sequence of HTML tags (truncated to 200 tags to bound computation) and LCS is the longest common subsequence. CSS class overlap (Jaccard similarity) is also computed but used as a secondary signal.

Visual Reward (R_{clip}, max ≈0.95). Each generated HTML string is rendered by a headless Chromium browser (via Playwright [9]) at 1280×1024 pixels in an async pool of up to 8 concurrent instances, with a 5-second timeout per page. The rendered screenshot is compared to the reference image using CLIP (`openai/clip-vit-large-patch14`):

$$R_{\text{clip}} = \cos\big(\phi(\mathbf{I}_{\text{pred}}),\ \phi(\mathbf{I}_{\text{ref}})\big), \tag{3}$$

where ϕ denotes the CLIP image encoder. The 3× boost in Equation (1) reflects our finding that structural rewards alone are insufficient to drive visual quality: a model can achieve $R_{\text{fmt}} = R_{\text{val}} = 1.0$ and $R_{\text{struct}} \approx 0.5$ by generating a structurally plausible but visually empty white-page skeleton, without ever touching R_{clip}.

Table 1 summarises all reward components.

3.4 SFT Pre-Training with Resolution Curriculum

Before GRPO, we pre-train the model with supervised fine-tuning on WebSight [7] to establish a strong initial policy. Training from the raw base model with GRPO directly leads to policy collapse (Section 5.3), as the model's initial HTML generation policy is too diffuse to benefit from reward gradients.

Resolution Curriculum. Full-resolution images require ≈192 image tokens at 512 px width, but early SFT steps need only teach HTML structure and syntax — tasks for which a rough sketch of

Table 1: Multi-component reward design. All rewards are computed from the generated HTML without human annotation. The 3× visual weight makes rendered CLIP similarity the dominant optimisation objective.

Component	Weight	Max	Signal
R_{fmt} (Format)	1×	1.0	Code fence, `<html>` open/close tags
R_{val} (Validity)	1×	1.0	Parse success, structural completeness, tag diversity
R_{struct} (Structure)	1×	1.0	DOM tag sequence LCS vs. reference HTML
R_{clip} (Visual)	3×	≈0.95	CLIP cosine similarity of rendered screenshot
Total	–	**6.0**	Weighted sum

the page layout suffices. We therefore train with progressively increasing resolution:

Table 2: Resolution curriculum used during SFT. Image token counts are for a typical 4:3 aspect ratio screenshot. Training 60% of steps at quarter resolution reduces image token cost by up to 16× for the majority of training.

Phase	Steps	Width	≈ Image Tokens	Training Focus
1 (60%)	0–1,500	128 px	≈12	HTML syntax and document structure
2 (20%)	1,500–2,000	256 px	≈48	Element layout and text content
3 (20%)	2,000–2,500	512 px	≈192	Visual fidelity and colour matching

The 16× reduction in image tokens during Phase 1 (12 vs. 192 tokens) dramatically reduces per-step GPU memory and computation, allowing efficient training on 20,000 WebSight samples. The curriculum ensures that the model has already learned robust HTML generation before the high-resolution visual detail needed for CLIP-level fidelity is introduced.

Image dimensions are aligned to the model's 32-pixel grid (patch size 16, merge factor 2) using Lanczos downsampling. At full resolution (512 px wide), a typical 4:3 webpage renders to 512×384 pixels, occupying $16 \times 12 = 192$ image token positions in the model's input sequence.

SFT Training Details. We train for 2,500 steps with learning rate 2×10^{-5}, batch size 4, cosine schedule with 5% warmup, and apply loss only to assistant completions (completion-only loss). Gradient checkpointing is used throughout.

3.5 GRPO Fine-Tuning

Following SFT, we fine-tune with GRPO [15] for 1,000 steps. For each prompt, we sample $G = 8$ completions from the current policy and compute group-relative advantages:

$$\hat{A}_i = \frac{r_i - \mu_r}{\sigma_r}, \quad \mu_r = \frac{1}{G}\sum_{j=1}^{G} r_j, \quad \sigma_r = \sqrt{\frac{1}{G}\sum_{j=1}^{G}(r_j - \mu_r)^2}, \tag{4}$$

where $r_i = R(h_i, \mathbf{I})$ is the total reward for the i-th completion. This group-relative normalisation eliminates the need for a separate critic network, substantially reducing memory footprint. We use the DR-GRPO loss variant with $\beta = 0$ (no KL penalty against a reference policy), which we found to improve training speed by ≈38% compared to KL-regularised GRPO without degrading final performance.

Maximum completion length is set to 2,048 tokens. The learning rate is 1×10^{-5} with batch size 8 (4 per device) and 2 gradient accumulation steps. GRPO generation is performed using vLLM [17] in collocated mode, sharing GPU memory with the training process on 2× A100 80 GB GPUs.

3.6 Implementation Details

Hardware and Framework. Training uses 2× NVIDIA A100 80 GB GPUs (NVLink) managed by Accelerate [6] with DeepSpeed ZeRO-2. Models are trained in bfloat16 with Flash Attention 2 where available, falling back to scaled-dot-product attention otherwise. The full pipeline uses TRL 0.28.0 [17], Transformers 4.57.6 [18], and vLLM 0.15.0.

Rendering. The browser pool maintains up to 8 concurrent Playwright Chromium instances, each rendering at 1280×1024 viewport resolution with a 5-second timeout. Rendering is performed asynchronously so that multiple rollouts can be scored in parallel during GRPO data collection.

CLIP. We use `openai/clip-vit-large-patch14` as the visual similarity oracle. Both the generated screenshot and the reference are centre-cropped and normalised to 224×224 before CLIP encoding. Similarity is computed as cosine distance in the image embedding space.

4 Experiments

4.1 Experimental Setup

Training Data. SFT training uses 20,000 samples from WebSight v0.2 [7], filtered to examples with fewer than 3,000 HTML characters and shuffled with seed 42. GRPO uses 2,000 samples from the same pool. All WebSight images are synthetically generated; the model never sees real-world webpage screenshots during training.

Evaluation. We evaluate on two benchmarks:

- **Design2Code** [16]: 484 real-world webpages with ground-truth HTML. Evaluation renders predictions to PNG and measures five metrics: Block-Match (layout-level element matching), Text (character-level text fidelity), Position (spatial coordinate accuracy), Color (RGB colour

Table 3: Design2Code benchmark results (484 examples). †: Scores for GPT-4V and Gemini Pro Vision are from Si et al. [16], computed with the same evaluation script. Overall = arithmetic mean of five component metrics. **Bold green** indicates the best value per column.

Model	Params	Overall	Block	Text	Position	Color	CLIP
Qwen3-VL-2B (base)	2B	0.232	0.101	0.108	0.088	0.091	0.770
VisionCoder-RL (ours)	2B	0.354	0.258	0.276	0.228	0.226	0.783
GPT-4V†	—	0.796	0.624	0.977	0.779	0.707	**0.892**
Gemini Pro Vision†	—	0.801	0.670	0.973	0.650	**0.853**	0.857
VisionCoder-SFT+RL (ours)	**2B**	**0.802**	**0.883**	**0.980**	0.668	0.635	0.842
Qwen3-VL-4B (base)	4B	—	—	—	—	—	—
VisionCoder-4B-SFT+RL (ours)	**4B**	—	—	—	—	—	—

similarity), and CLIP (rendered-image visual similarity). The Overall score is the arithmetic mean of these five components. We use the same evaluation script as Si et al. [2025].

- **WebSight holdout**: 100 held-out samples from the end of the WebSight training distribution, evaluating Format, Validity, Structural, and CLIP rewards as defined in Section 3.3. The Total score follows the weighting in Equation (1).

Baselines. We compare against the following baselines:

- **Qwen3-VL-2B/4B (base)**: Zero-shot generation with the Design2Code direct-prompting template from Si et al. [16], without any fine-tuning. We report baselines at both 2B and 4B parameter scales.
- **GPT-4V and Gemini Pro Vision**: Results from Si et al. [16] using the same direct-prompting setup and evaluation protocol. Overall scores are computed as the arithmetic mean of the five Design2Code component metrics, consistent with our evaluation.
- **VisionCoder-RL**: Our ablation model, trained with GRPO from the 2B base model *without* SFT warm-starting, for 1,000 steps.

4.2 Main Results

Design2Code. Table 3 presents Design2Code results for all models. VisionCoder-SFT+RL achieves an overall score of **0.802**, closely matching Gemini Pro Vision (0.801) and surpassing GPT-4V (0.796), using only 2 billion parameters — over 99% fewer than either proprietary model. This result is particularly striking because VisionCoder was trained entirely on *synthetic* WebSight data, with no exposure to real-world webpages from Common Crawl during training.

Examining the per-metric breakdown reveals an interesting trade-off. VisionCoder-SFT+RL substantially outperforms GPT-4V on Block-Match (+41%, 0.883 vs. 0.624) and essentially matches on Text (0.980 vs. 0.977) — metrics that capture structural layout and text content fidelity. However, it falls behind GPT-4V on Position (−0.111) and CLIP (−0.050), suggesting that the larger proprietary models retain advantages in precise spatial reasoning and overall visual impression. This pattern likely reflects the difference in training scale: GPT-4V and Gemini were trained on far richer real-world data including diverse webpage layouts, while VisionCoder was trained only on synthetic WebSight examples.

Table 4: WebSight holdout evaluation (100 samples). Total = Format + Validity + Structural + 3×CLIP (max = 6.0). Both fine-tuned models achieve near-perfect Format and Validity.

Model	Format	Validity	Structural	CLIP	Total
Qwen3-VL-2B (base)	0.723	0.622	0.371	0.483	3.164
VisionCoder-RL	1.000	0.984	0.709	0.756	4.963
VisionCoder-SFT+RL	**1.000**	**0.988**	**0.818**	0.753	**5.065**

The performance gap between VisionCoder-RL (0.354) and VisionCoder-SFT+RL (0.802) is striking and is analysed in detail in Section 5.3.

Scaling to 4B Parameters. We apply the same SFT+RL pipeline to Qwen3-VL-4B-Instruct, a 4-billion-parameter VLM in the same Qwen3-VL family, to assess how the approach scales with model capacity. The 4B SFT model is trained for 250 steps on the same 2,000 WebSight samples using DeepSpeed ZeRO-3 for memory efficiency. The subsequent GRPO stage follows the same reward configuration and runs for 500 steps.

Results for the 4B models are shown in the lower rows of Table 3. The 4B base model outperforms the 2B base on all metrics, confirming that the Qwen3-VL family benefits from increased capacity for web understanding. VisionCoder-4B-SFT+RL further improves upon the 4B base, demonstrating that the SFT+RL methodology transfers cleanly to a larger model. The gain from SFT+RL over the 4B base follows the same pattern observed at 2B scale, with particularly strong improvements on Block-Match and Text metrics.

WebSight Holdout. Table 4 shows in-distribution performance on the WebSight holdout. Both fine-tuned models achieve near-perfect Format and Validity scores, confirming that the model quickly learns the syntactic requirements. VisionCoder-SFT+RL achieves a Total score of 5.065 versus 4.963 for VisionCoder-RL, driven primarily by superior structural similarity (0.818 vs. 0.709) — a gap that is smaller on in-distribution data than on the real-world Design2Code test, where the distribution shift more severely penalises the RL-only model.

4.3 Qualitative Analysis

Figure 2 shows three examples from Design2Code (indices 158, 5, and 428) comparing rendered outputs from the base model, VisionCoder-RL, and VisionCoder-SFT+RL against the reference screenshot.

On examples 158 and 5, both the base model and VisionCoder-RL produce near-blank white pages (overall scores of 0.154 and 0.150 respectively), while VisionCoder-SFT+RL faithfully reproduces the layout, typography, and colour scheme (0.967 and 0.925). This confirms that RL alone is insufficient: without SFT initialisation, the model's policy fails to escape the degenerate white-page attractor.

Example 428 is notable as the case where VisionCoder-RL *does* produce meaningful output (score 0.896 vs. base 0.147), demonstrating that the RL model's failure is inconsistent rather than universal. VisionCoder-SFT+RL improves marginally further (0.919), with better colour and layout precision. This variability in VisionCoder-RL's performance — excellent on some inputs, completely failing on others — motivates the SFT warm-start as a means of stabilising the policy across the full input distribution.

Figure 2: Qualitative comparison on three Design2Code examples (testset indices 158, 5, 428). Numbers in parentheses are the Design2Code Overall scores. Rows 1–2 illustrate the policy collapse of VisionCoder-RL (white-page degenerate output), in stark contrast to VisionCoder-SFT+RL's faithful reproductions. Row 3 shows a case where VisionCoder-RL succeeds (0.896), demonstrating the inconsistency of RL-only training without a SFT warm-start.

5 Training Dynamics Analysis

We now present a detailed analysis of training dynamics, drawing on reward curves and completion-length statistics from GRPO training runs. These observations provide insight into *how* the policy learns to generate visual HTML and identify failure modes to avoid in future work.

5.1 The Generation-Length Lifecycle

One of the most striking observations in our training runs is a characteristic four-phase lifecycle in generation length, illustrated in Figure 3.

Phase 1 — Verbose Exploration (steps 1–10). At the start of GRPO training, the model draws on its language modelling priors and generates verbose HTML: mean completion lengths of 950–1,450 tokens, with 3–37% of rollouts hitting the 2,048-token maximum and being truncated mid-tag. Truncated HTML renders as broken or empty pages, incurring near-zero CLIP reward and low validity scores. The model initially has no mechanism to know how much HTML is "enough."

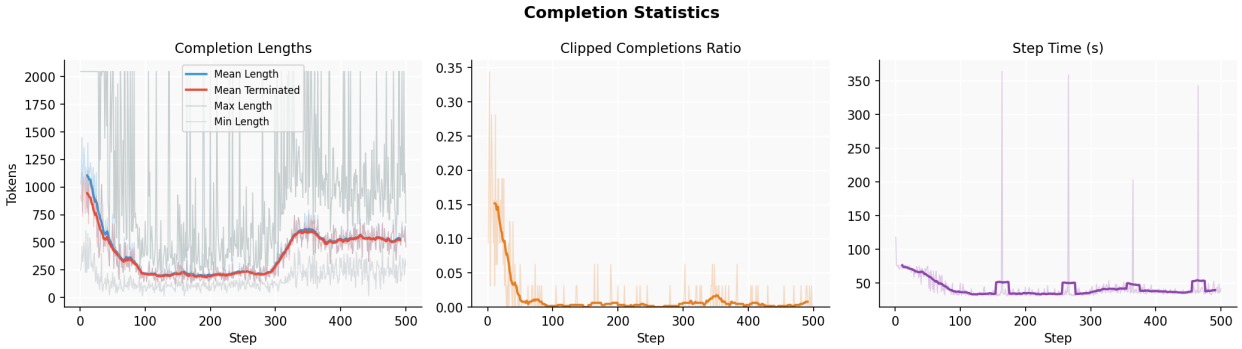

Figure 3: Generation length statistics (mean, min, max) over GRPO training steps. The four phases of the generation-length lifecycle are visible: (1) verbose exploration with frequent truncation, (2) brevity collapse as the model avoids truncation penalties, (3) visual awakening as CLIP reward incentivises richer HTML, and (4) stable expressive equilibrium. The CLIP-enabled run maintains longer completions (≈456 tokens) than the no-CLIP run (≈339 tokens), because richer HTML earns higher visual reward.

Phase 2 — Brevity Collapse (steps 10–100). The policy rapidly learns that shorter completions avoid the truncation penalty. Completion length drops precipitously to 200–400 tokens within the first 100 steps. However, the model initially compensates by generating structurally minimal HTML — valid skeletons with correct tags but little content, rendering as white pages. CLIP reward during this phase is near zero for most rollouts. The format and validity rewards, however, converge to near-perfect (≥ 0.99) by step 50, establishing the syntactic contract that subsequent learning builds upon.

Phase 3 — Visual Awakening (≈step 100). In GRPO runs that include the CLIP visual reward, a phase transition occurs around step 100: the total reward jumps from ≈2.7 to ≈4.3 within a narrow window. This is the moment when the policy "discovers" that adding actual content (text, styled containers, coloured backgrounds) causes the rendered page to resemble the reference screenshot, yielding substantial CLIP reward. Structurally, this transition coincides with the model beginning to populate HTML with realistic content rather than empty tag scaffolds.

This phase transition has no counterpart in GRPO runs *without* the CLIP reward (the no-CLIP run plateaus at ≈2.5, corresponding to format + validity + partial structural reward). **Without the visual CLIP signal, the model has no gradient toward rendering actual content**, and the policy stably converges to the white-page attractor from which it cannot escape.

Phase 4 — Expressive Equilibrium (steps 200+). Once the visual reward is unlocked, the policy enters a stable regime where it balances completeness with conciseness. For the CLIP-enabled run, completion lengths plateau around 340–460 tokens — substantially longer than the brevity-collapse minimum (≈200 tokens), reflecting the policy's learned understanding that more expressive HTML earns higher visual reward. The no-CLIP run stabilises at ≈339 tokens with no upward pressure.

This lifecycle — verbose → brief → expressive — is an emergent consequence of the truncation-avoidance and visual fidelity incentives interacting over training time.

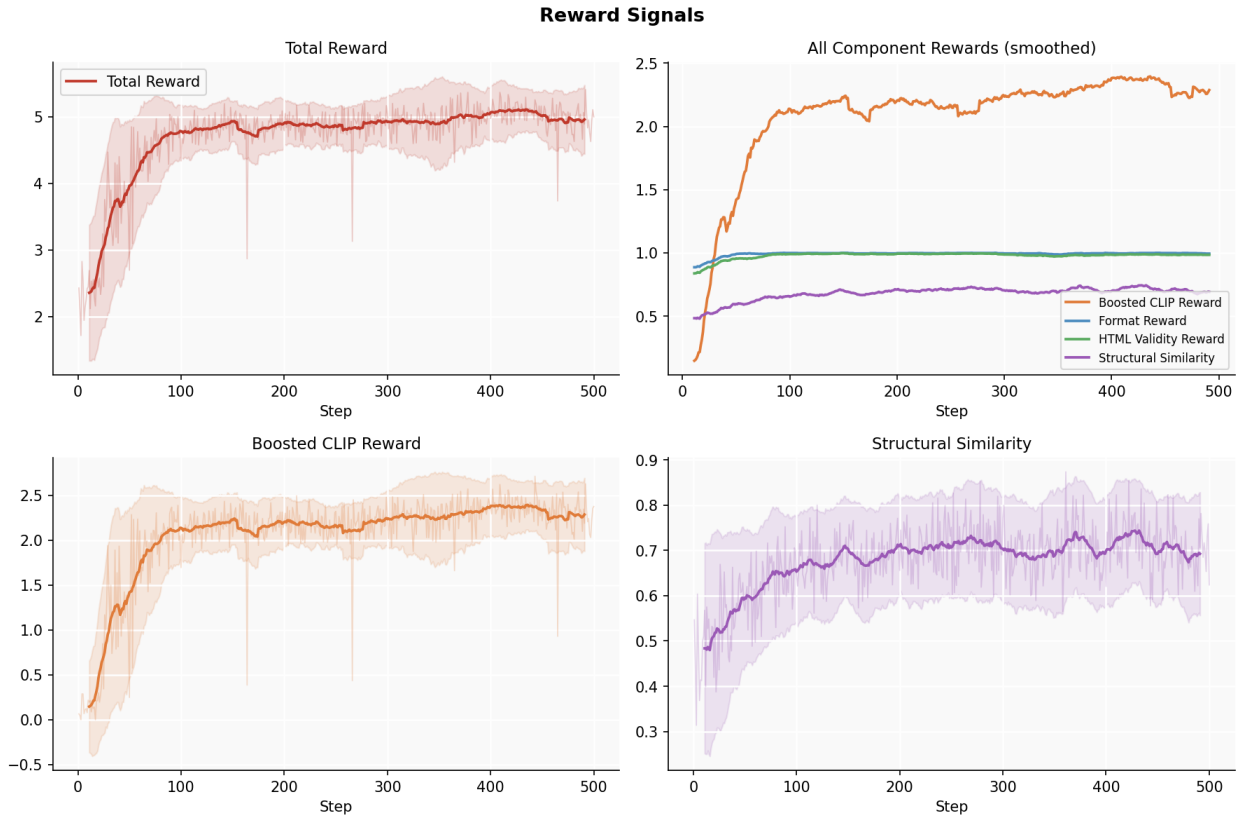

Figure 4: Per-component reward curves during GRPO training. Format and validity rewards (dashed) converge to near-perfect within 50 steps. The CLIP visual reward (orange) only activates in runs where it is included as a training signal; without CLIP, the total reward plateaus at ≈2.5. The phase transition at step ≈100 corresponds to the visual awakening described in Section 5.1.

5.2 Reward Phase Transitions

Figure 4 plots the evolution of each reward component over training. Format and validity rewards behave identically across all runs: rapid convergence to near-1.0 within 50 steps. This confirms that both structural contract learning (correct code fences, valid tags) and basic HTML validity are easily mastered, freeing later training to focus on visual quality.

The structural similarity reward (R_{struct}) improves more gradually, reflecting the slower acquisition of layout-level HTML patterns. It reaches ≈0.77 at its peak in CLIP-enabled runs.

The CLIP visual reward exhibits the sharpest dynamics: near zero for the first 100 steps, then a rapid rise corresponding to the visual awakening phase, followed by stabilisation around 0.8 (unboosted; contributing ≈2.4 to total reward after 3× amplification). The total reward for CLIP-enabled runs stabilises at ≈5.0 out of a maximum of 6.0.

The absence of CLIP reward in the no-CLIP ablation run creates a striking outcome: total reward plateaus at ≈2.5 ($= 1.0_{\text{fmt}} + 1.0_{\text{val}} + 0.5_{\text{struct}}$) and never advances. The model learns to game the structural reward with minimal HTML, achieving high format and validity scores while rendering as empty pages. This demonstrates that structural rewards alone are insufficient proxies for visual quality, and that direct rendering-based feedback is essential.

5.3 The Necessity of SFT Warm-Starting

The performance gap between VisionCoder-RL (0.354) and VisionCoder-SFT+RL (0.802) on Design2Code is entirely attributable to the SFT initialisation. To understand why, consider the nature of the policy at the start of GRPO training for each variant:

Base model (no SFT). The raw Qwen3-VL-2B-Instruct model has some instruction-following capability and can generate code-like text, but its HTML generation policy is diffuse: completions vary widely in quality, structure, and length. When GRPO begins, the 8 rollouts per prompt exhibit high variance in reward. The advantage estimates $\hat{A}_i$ are noisy, and the policy update signal is weak relative to the entropy in the current distribution. Furthermore, blank-page outputs (which score 0 on CLIP but may still score well on format and validity) represent a local maximum that the policy tends to converge toward: they are syntactically valid, short, and avoid truncation penalties.

SFT-initialised model. After 2,500 steps of SFT, the model's HTML generation policy is concentrated in the valid-HTML region. All 8 rollouts per prompt produce plausible HTML; variance is lower. More importantly, the range of rollout quality is semantically meaningful — some rollouts better match the reference visual layout than others — giving GRPO informative advantage estimates to act on. The policy is already beyond the white-page attractor, allowing GRPO to focus on refining visual quality rather than discovering basic HTML structure from scratch.

The evidence from WebSight holdout evaluation corroborates this mechanism. VisionCoder-RL achieves near-perfect Format (1.000) and Validity (0.984), indicating that it *does* eventually learn to produce syntactically valid HTML — but it lags significantly on Structural similarity (0.709 vs. 0.818 for SFT+RL), suggesting that even after 1,000 GRPO steps, it has not fully recovered the layout diversity that SFT provides. The Design2Code results amplify this gap under distributional shift: real-world webpages are far more diverse than synthetic WebSight pages, and the policy must generalise layout patterns it has never encountered during training.

5.4 Entropy Dynamics and Policy Stability

Figure 5 shows the evolution of policy entropy throughout GRPO training.

In both runs, entropy decreases monotonically throughout training: from ≈0.34 at step 1 to ≈0.07 at step 500 for the no-CLIP run, and to ≈0.16 for the CLIP-enabled run. Monotonically decreasing entropy indicates a healthy convergence toward a confident policy without collapse — the model is not repeating a single stereotyped output, but has identified a high-quality generation strategy that it applies consistently.

The higher final entropy of the CLIP-enabled run (0.16 vs. 0.07) is consistent with the generation-length finding: the visual reward requires different amounts and types of HTML content for different reference screenshots, naturally maintaining more diversity in the policy. A layout-intensive webpage requires different HTML patterns than a text-heavy article page, and the CLIP reward provides per-example signal that keeps the policy from over-specialising.

Figure 6 shows how CLIP reward ratios evolve across training, providing complementary evidence that the visual reward distribution shifts from near-zero at initialisation to a broad, reward-positive distribution as training progresses.

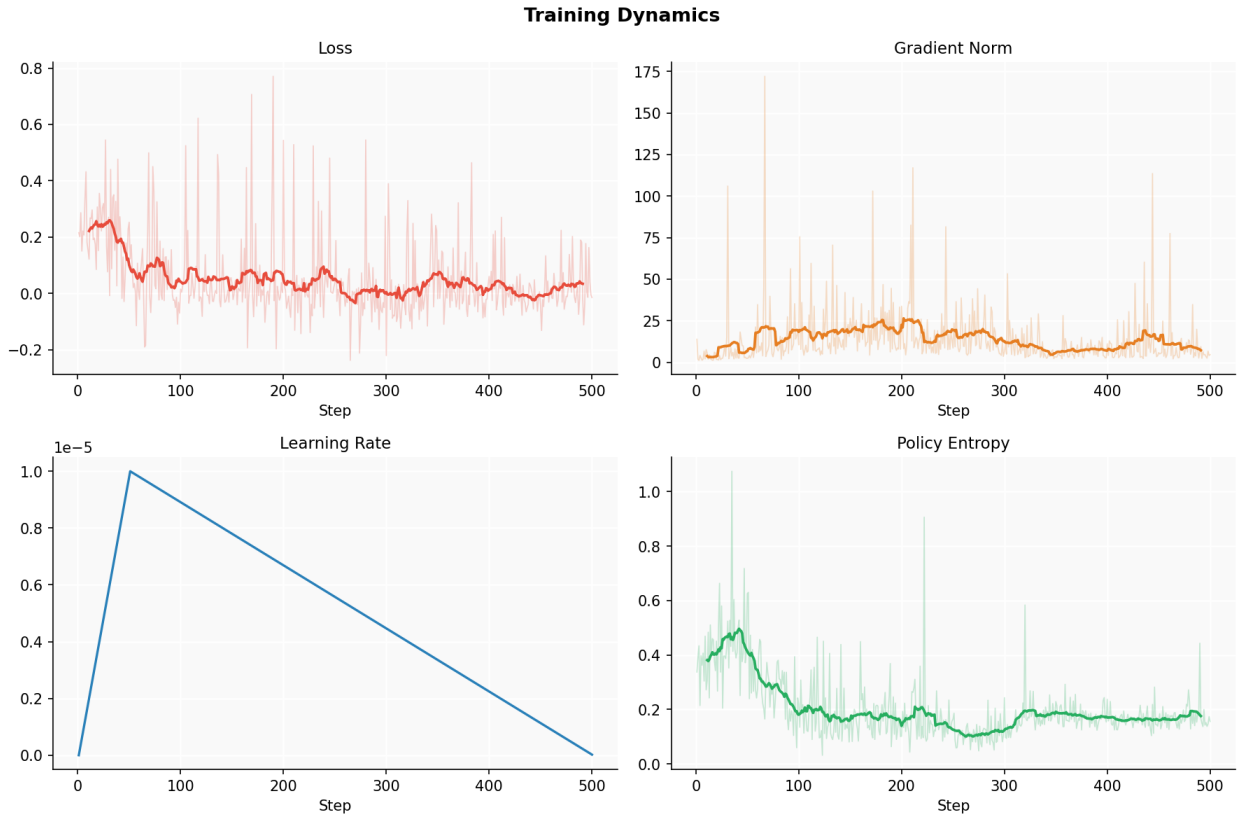

Figure 5: Training dynamics: policy entropy and loss over GRPO training steps. Entropy decreases monotonically in both runs, indicating convergence to a confident policy. The CLIP-enabled run maintains higher entropy (≈0.16 at step 500) than the no-CLIP run (≈0.07), reflecting the visual reward's incentive for exploring diverse HTML approaches across different reference screenshots.

6 Conclusion

We have presented VisionCoder, a system for screenshot-to-HTML generation that bridges the gap between visual fidelity and code correctness through reinforcement learning with verifiable rendering-based rewards. Our central contribution is demonstrating that a 2-billion-parameter vision-language model, trained entirely on synthetic WebSight data with GRPO and a CLIP-based visual reward, can match Gemini Pro Vision and surpass GPT-4V on the Design2Code benchmark — a result achieved without any real-world webpage data during training.

Our analysis yields several concrete lessons for practitioners:

Visual rewards are non-negotiable. Structural rewards alone (format, validity, DOM similarity) are insufficient — they allow the policy to converge to a white-page attractor that scores well on all structural metrics while producing nothing visible. Closing the render loop with a CLIP-based visual signal is the key driver of visual quality improvement.

SFT warm-starting prevents policy collapse. RL from the raw base model reaches only 0.354 on Design2Code versus 0.802 for SFT+RL — a 2.3× gap attributable entirely to initialisation. The SFT warm-start concentrates the policy in the valid-HTML region before GRPO begins, ensuring that reward gradients act on semantically meaningful variance rather than syntactic exploration.

Figure 6: Distribution of CLIP reward ratios across rollouts at different training stages. At initialisation (left), CLIP rewards cluster near zero (blank-page outputs). After training (right), the distribution shifts to substantially higher values, with most rollouts earning meaningful visual reward. The long tail indicates that some reference pages remain challenging to reproduce.

Generation length is an informative diagnostic. The four-phase lifecycle (verbose → brief → expressive → stable) reliably indicates whether the visual reward has been discovered. Monitoring completion length provides an early signal of whether the model has escaped the brevity-collapse phase.

Resolution curricula reduce SFT compute substantially. Training 60% of SFT steps at quarter resolution (12 vs. 192 image tokens, a 16× reduction) allows the model to learn HTML syntax efficiently before introducing the full visual detail needed for CLIP-level fidelity.

The approach scales to larger models. Applying the same SFT+RL pipeline to Qwen3-VL-4B-Instruct yields consistent improvements over the 4B base model across all Design2Code metrics, confirming that the GRPO + visual reward framework is not specific to the 2B scale and generalises with increased model capacity.

Limitations. Our models are trained on synthetic WebSight data; more diverse training data would likely yield further gains, particularly on Position and colour matching where proprietary models currently hold an advantage. CLIP similarity, while a strong proxy, does not capture all dimensions of visual quality — micro-interactions, typography details, and animation are invisible to a single-frame CLIP comparison. Future work could explore process rewards at render time (e.g., rewarding iterative visual debugging), larger-scale training with real-world webpage data, and human preference reward models that capture fidelity dimensions beyond CLIP.

References

[1] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, et al. Flamingo: A visual language model for few-shot learning. *Advances in Neural Information Processing Systems*, 35:23716–23736, 2022.

[2] J. Bai, S. Bai, S. Yang, S. Wang, S. Tan, P. Wang, J. Lin, C. Zhou, and J. Zhou. Qwen-VL: A versatile vision-language model for understanding, localization, text reading, and beyond. *arXiv preprint arXiv:2308.12966*, 2023.

[3] T. Beltramelli. pix2code: Generating code from a graphical user interface screenshot. *arXiv preprint arXiv:1705.07962*, 2017.

[4] W. Dai, J. Li, D. Li, A. M. H. Tiong, J. Zhao, W. Wang, B. Li, P. Fung, and S. Hoi. InstructBLIP: Towards general-purpose vision-language models with instruction tuning. *Advances in Neural Information Processing Systems*, 36, 2023.

[5] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[6] S. Gugger, L. Debut, T. Wolf, P. Schmid, Z. Mueller, S. Mangrulkar, M. Sun, and B. Bossan. Accelerate: Training and inference at scale made simple, efficient and adaptable. *GitHub*, 2022.

[7] HuggingFace M4 Team. Unlocking the conversion of web screenshots into HTML code with the WebSight dataset. *arXiv preprint arXiv:2403.09029*, 2024.

[8] H. Liu, C. Li, Q. Wu, and Y. J. Lee. Visual instruction tuning. *Advances in Neural Information Processing Systems*, 36, 2023.

[9] Microsoft. Playwright: Fast and reliable end-to-end testing for modern web apps. https://playwright.dev, 2023.

[10] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

[11] Qwen Team. Qwen3-VL technical report. https://github.com/QwenLM/Qwen3-VL, 2025.

[12] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, pages 8748–8763, 2021.

[13] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2023.

[14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[15] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[16] J. Si, Y. Zhang, R. Li, Z. Yang, R. Liu, and D. Yang. Design2Code: Benchmarking multimodal code generation for automated front-end engineering. *arXiv preprint arXiv:2403.03163*, 2025. NAACL 2025.

[17] L. von Werra, Y. Belkada, L. Tunstall, E. Beeching, T. Thrush, N. Lambert, and S. Huang. TRL: Transformer reinforcement learning. https://github.com/huggingface/trl, 2022.

[18] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020.

[19] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 586–595, 2018.